1

“Gang of Four” (GoF) Patterns

The Fundamental OO Design Patterns

Includes content adapted from *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, Addison-Wesley && *Java Design Patterns, 2022, Apress*



Previously on Design Patterns ...

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioral patterns

			Purpose	
		Creational	**Structural**	**Behavioral**
Scope	**Class**	1. Factory Method	1.1. Adapter(class)	1. Interpreter 2. Template Method
	Object	2. Singleton 3. Prototype 4. Builder 5. Abstract Factory	1.2. Adapter(object) 2. Proxy 3. Flyweight 4. Composite 5. Bridge 6. Facade 7. Decorator	3. Observer 4. Strategy 5. Command 6. Iterator 7. Memento 8. State 9. Mediator 10. Visitor 11. Chain of Responsibility



Previously on Design Patterns ...

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioral patterns

Today's topic

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	1. Factory Method	1.1. Adapter(class)	1. Interpreter 2. Template Method
	Object	2. Singleton 3. Prototype 4. Builder 5. Abstract Factory	1.2. Adapter(object) 2. Proxy 3. Flyweight 4. Composite 5. Bridge 6. Facade 7. Decorator	3. Observer 4. Strategy 5. Command 6. Iterator 7. Memento 8. State 9. Mediator 10. Visitor 11. Chain of Responsibility

1

Structural Patterns

How to define larger structures from smaller elements



Structural Patterns

- Focus on creating larger structures by grouping several smaller classes and objects, usually by means of composition or inheritance.
- These patterns improve the flexibility of efficiency of the program.



Structural Patterns

1. **Proxy**
2. Decorator
3. Adapter
4. Facade
6. Flyweight
7. Composite
8. Bridge

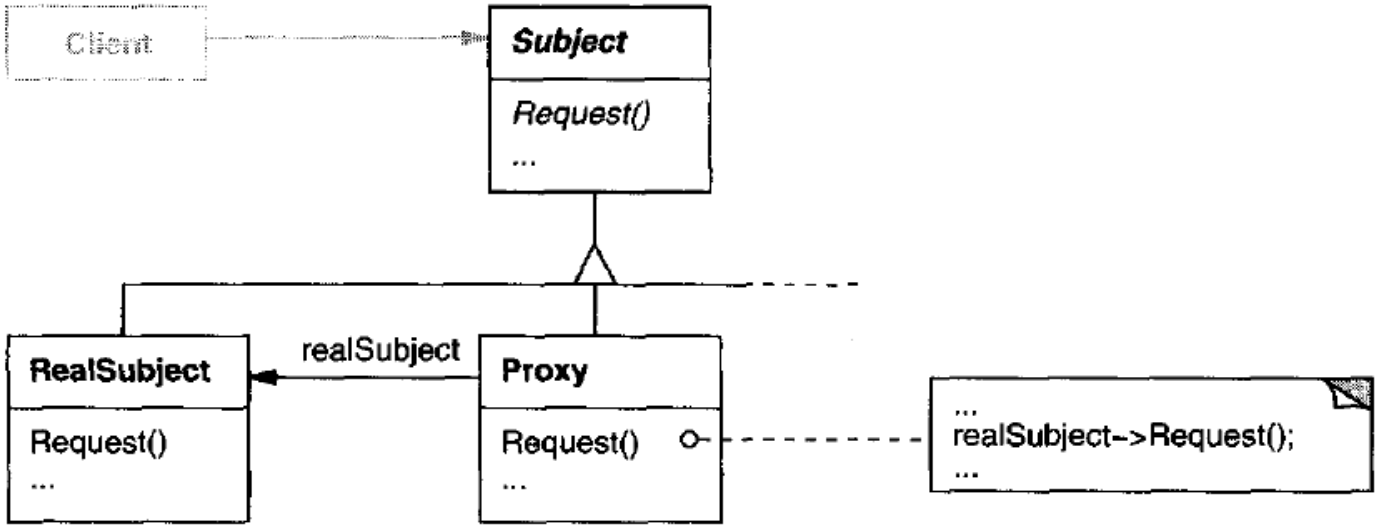


Proxy

- It provides a surrogate or placeholder for another object to control access to it.
- When a client interacts with a proxy object, it usually appears that it is directly talking to the actual object.
  - May be achieved by making proxy implement an interface of the proxied object.

Java API example: The package `java.rmi.*` uses proxies to allow an object in one JVM to invoke methods on an object residing in a different JVM.

Proxy


Client
Subject
Request()
...
RealSubject
Request()
...
realSubject
Proxy
Request()
...
...
realSubject->Request();
...



Proxy

- Different types of proxies can be described, depending on the intended purpose, e.g.:
  - **Remote Proxies** – These offer a local view of an object residing in a remote space.
  - **Virtual Proxies** – Used to perform optimizations, such as implementing on-demand initialization.
  - **Protection Proxies** – Used to implement access-control policies.
  - **Smart Reference** – Used to manage the lifecycle of the object, using techniques such as reference counting.



Structural Patterns

1. Proxy
2. **Decorator**
3. Adapter
4. Facade
6. Flyweight
7. Composite
8. Bridge



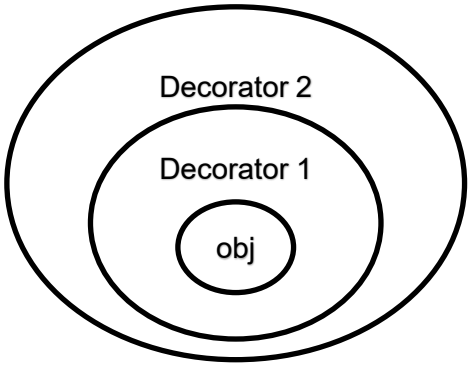
Decorator

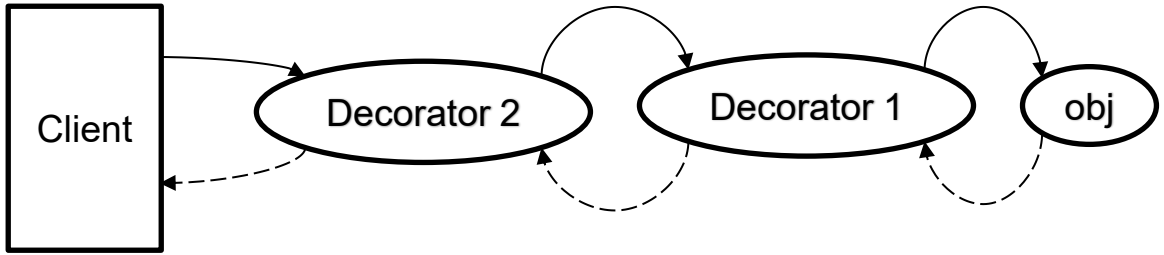
- It attaches additional responsibilities to an object dynamically, by wrapping the original object.
- Decorators provide a flexible alternative to subclassing for extending functionality.

Java API example: The IO stream classes. For example, a `java.io.BufferedOutputStream` object can decorate a `java.io.OutputStream` object.

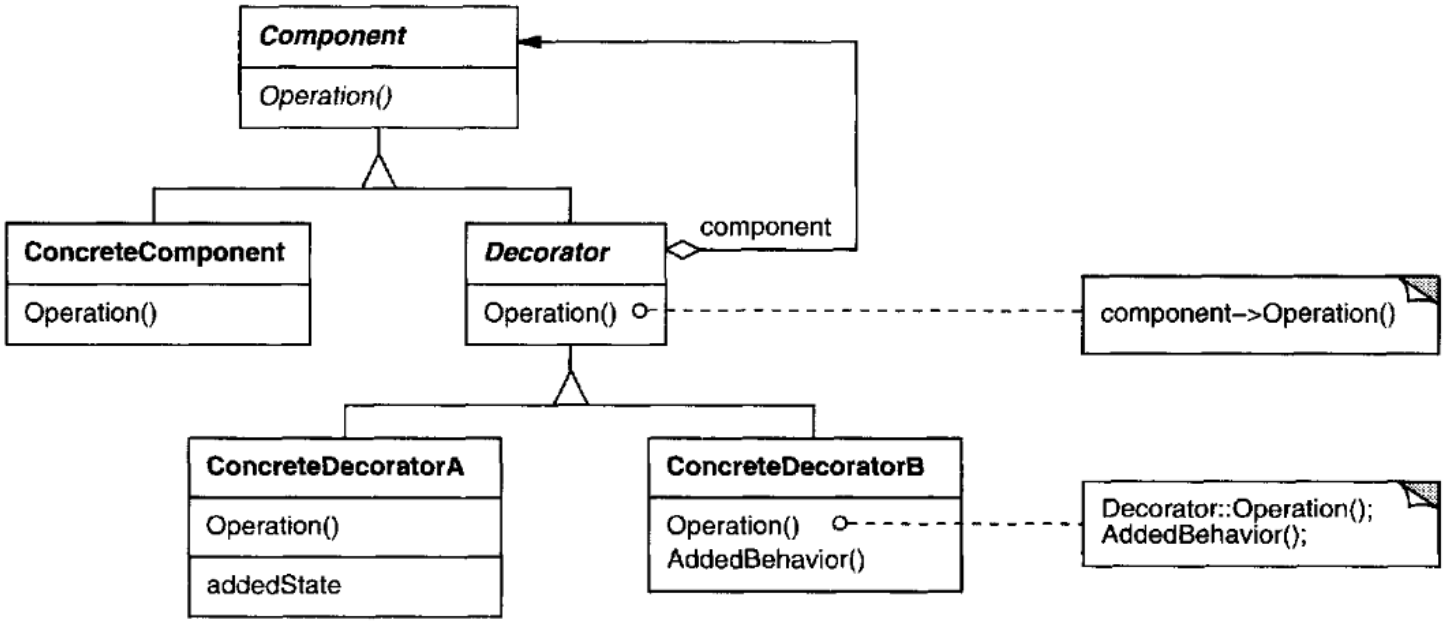
Decorator

- Decorators wrap around an internal object. Both implement a common interface, so that from a client's perspective, the existence of a decorator is not discernible.
  - Method invocations are delegated to the internal object, but are somehow modified to change the original behavior.

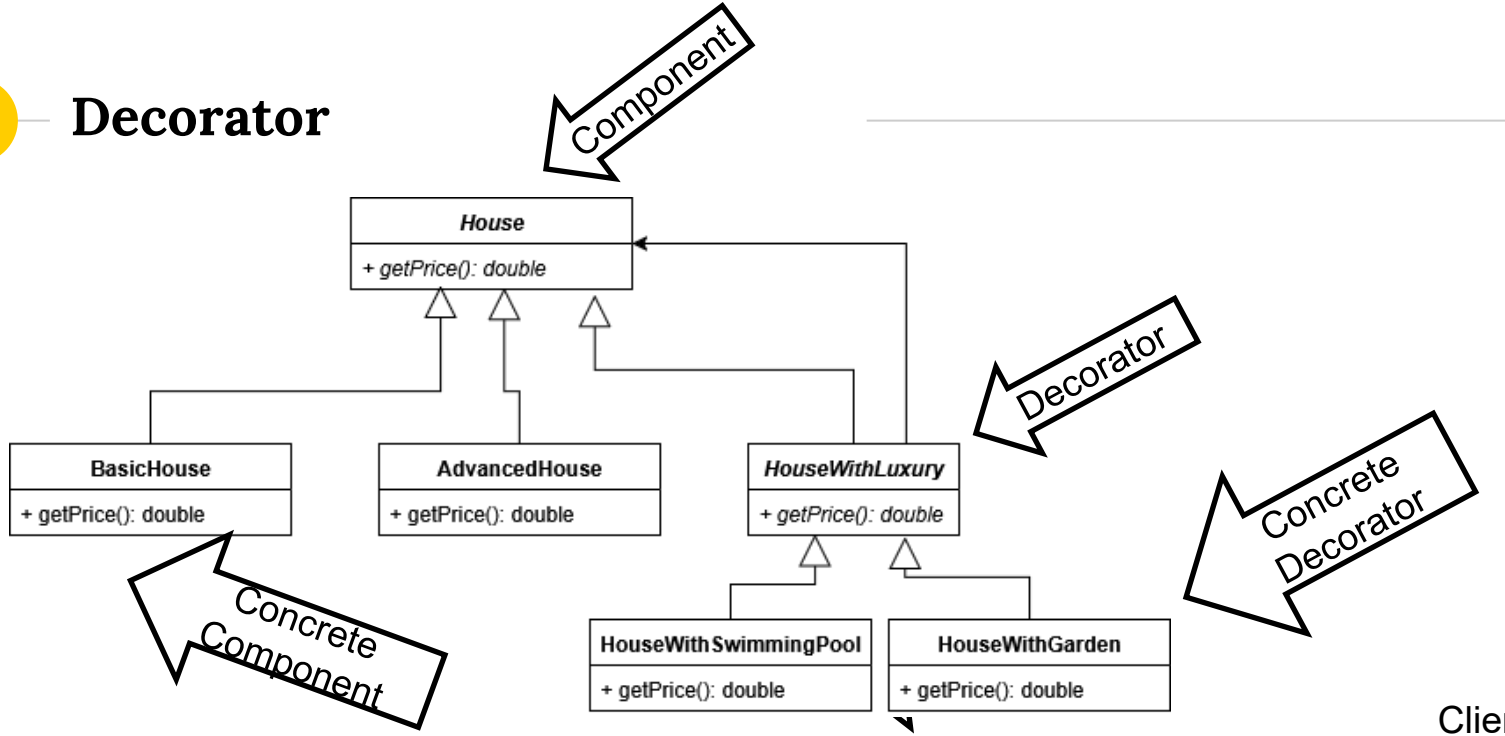




Decorator


Component
Operation()
ConcreteComponent
Operation()
Decorator
Operation()
component
component->Operation()
ConcreteDecoratorA
Operation()
addedState
ConcreteDecoratorB
Operation()
AddedBehavior()
Decorator::Operation();
AddedBehavior();

Decorator



Client Code

```
House home=new BasicHouse();
home=new HouseWithGarden(home);
Integer price=home.getPrice();
```



Decorator

Advantages:

- Decorated components are not modified, but their behaviour is extended (*open-closed principle*)
- Objects with complex variance in behaviour can be created dynamically. The use of static mechanisms such as inheritance to achieve the same effect would be inconvenient, because all the possibilities would have to be accounted for statically.



Structural Patterns

1. Proxy
2. Decorator
3. **Adapter**
4. Facade
6. Flyweight
7. Composite
8. Bridge



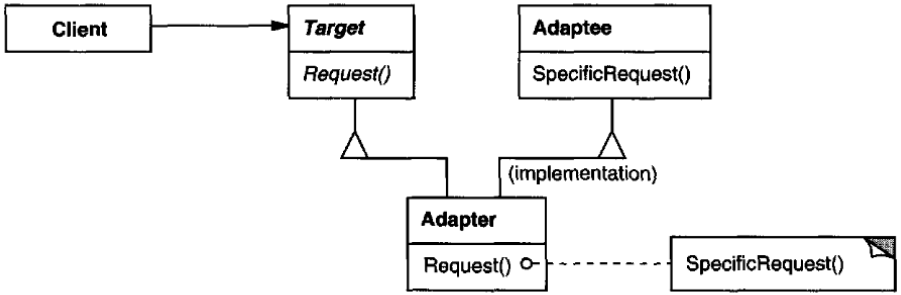
Adapter

- It converts the interface of a class into another interface that clients expect.
- The Adapter pattern lets classes work together that could not otherwise because of incompatible interfaces.
- There are two variations: *class* and *object* Adapters.

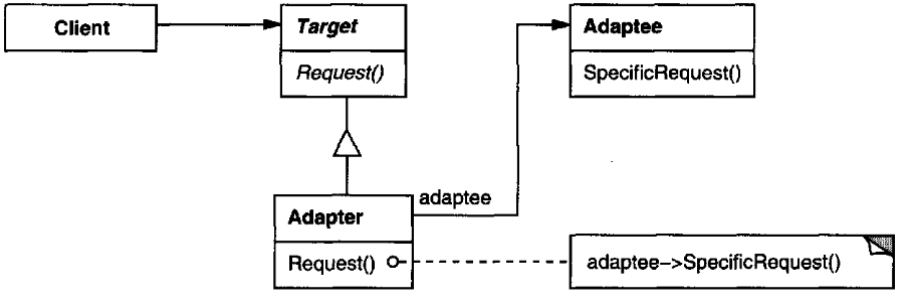
Java API example: The `java.io.InputStreamReader` and `java.io.InputStreamWriter` classes are examples of object adapters.

Adapter

Class Adapter

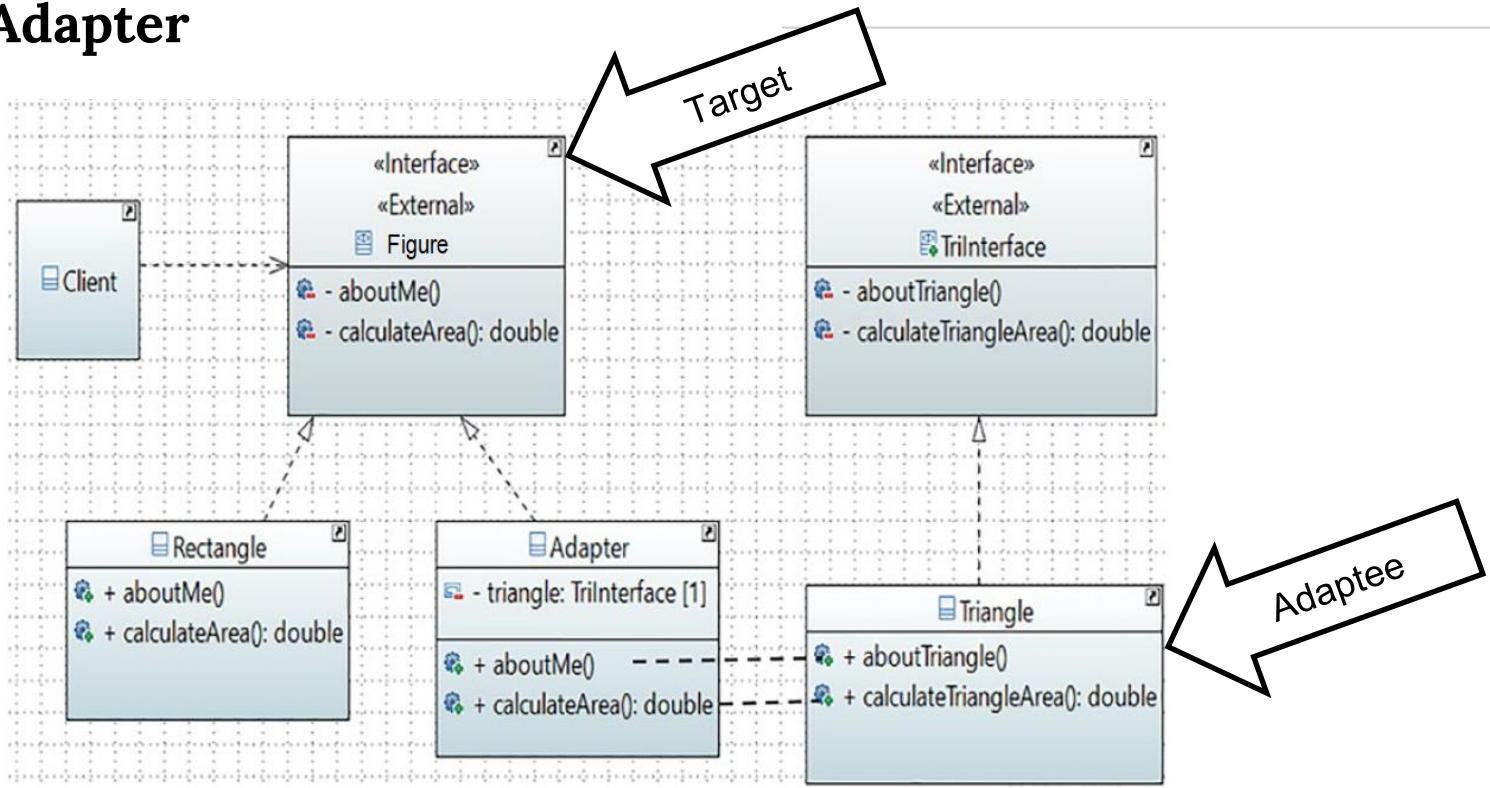


Object Adapter



- Also known as "two-way" adapter
  - Adapter can be used whenever any one of the `Target` and `Adaptee` interfaces are required.
- Requires availability for extension of the `Adaptee` interface and implementation of all its methods.
- Does not adapt a class *and* all its subclasses
- Most commonly used.
- Uses composition instead of inheritance for connecting with `Adaptee`.
- Adapts a class and all its subclasses.

Adapter


Target
«Interface»
«External»
Figure
- aboutMe()
- calculateArea(): double
«Interface»
«External»
TriInterface
- aboutTriangle()
- calculateTriangleArea(): double
Client
Rectangle
+ aboutMe()
+ calculateArea(): double
Adapter
- triangle: TriInterface [1]
+ aboutMe()
+ calculateArea(): double
Triangle
+ aboutTriangle()
+ calculateTriangleArea(): double
Adaptee



Structural Patterns

1. Proxy
2. Decorator
3. Adapter
4. **Facade**
6. Flyweight
7. Composite
8. Bridge

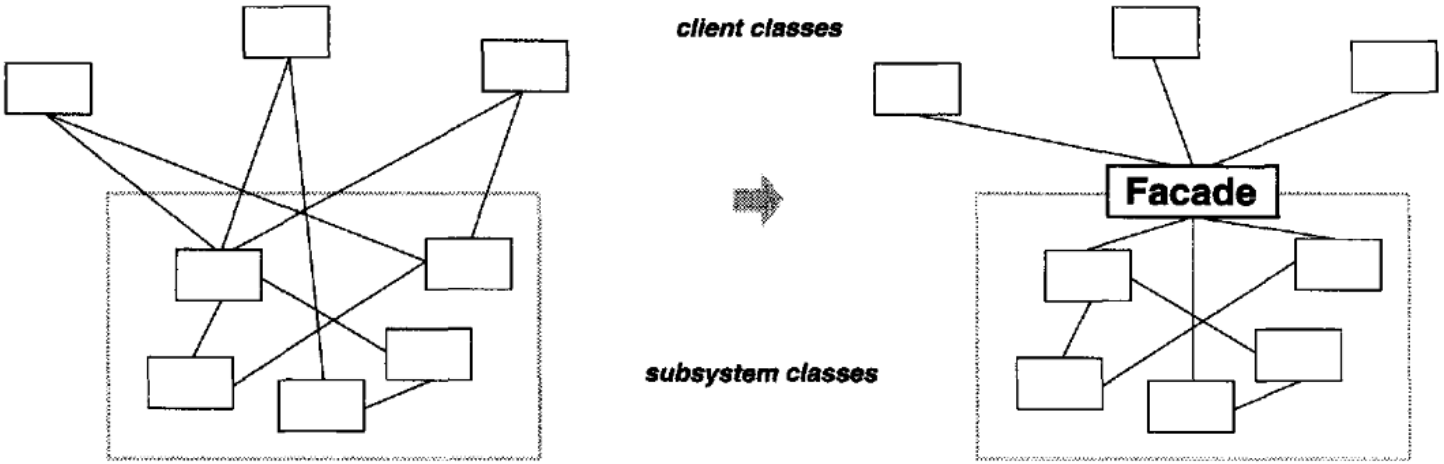


Facade

- It provides a unified interface to a set of interfaces in a subsystem.
- The Facade pattern defines a higher-level interface that makes the subsystem easier to use.
  - It promotes loose coupling between client and the subsystem

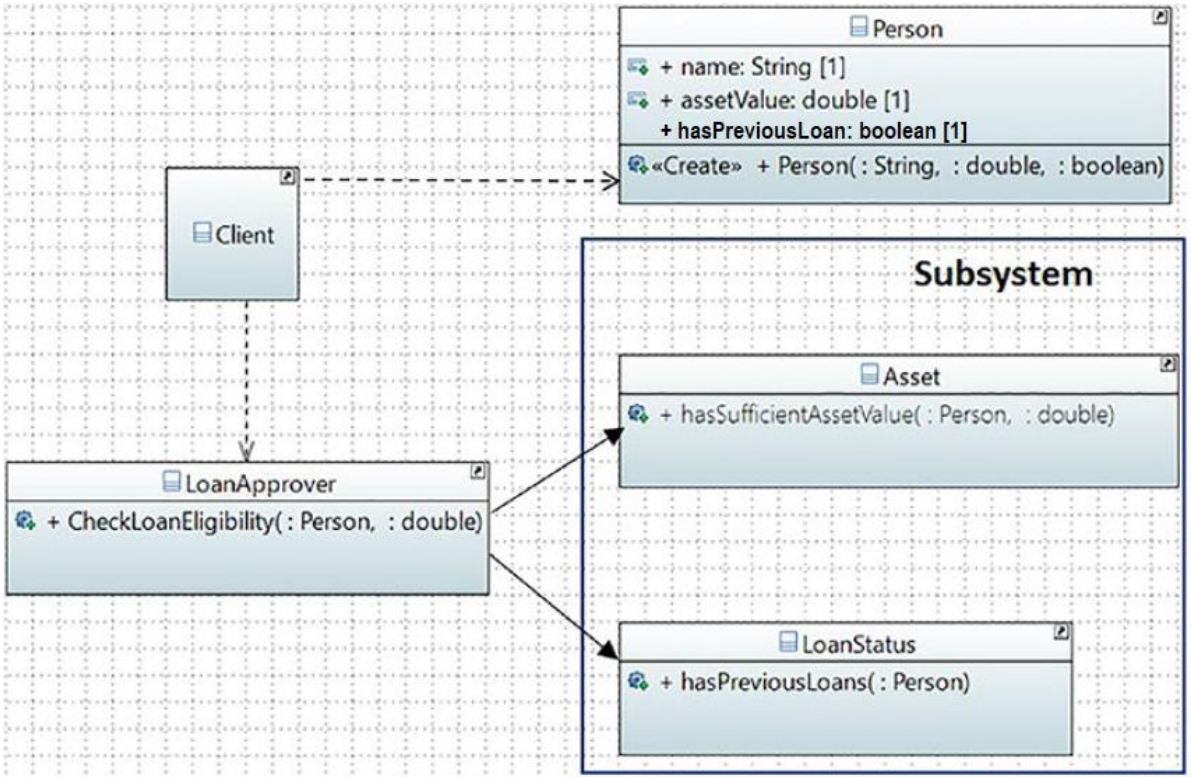
Java API example: The `java.net.URL` class is a facade for some operations of the stream library.

Facade



- Clients communicate with the subsystem by sending requests to the Facade.
- The subsystem classes handle most of the actual work. The Facade can do minor adaptations/translations.
- Usually, clients cannot interact directly with subsystem classes if a facade is used. In this case we have an *opaque* Facade. Otherwise, it's a *transparent* Facade.

Facade


Person
+ name: String [1]
+ assetValue: double [1]
+ hasPreviousLoan: boolean [1]
«Create» + Person(: String, : double, : boolean)
Client
Subsystem
Asset
+ hasSufficientAssetValue(: Person, : double)
LoanApprover
+ CheckLoanEligibility(: Person, : double)
LoanStatus
+ hasPreviousLoans(: Person)



Facade

Advantages:

- Shields clients from subsystem components, reducing the number of objects they interact with and making it easier to use.
- Promotes weak coupling between the client and the subsystem. This allows the subsystem to more easily change without negatively impacting the clients.
- Reduces build time (it is not necessary to recompile the entire universe when the project is rebuilt).



Structural Patterns

1. Proxy
2. Decorator
3. Adapter
4. Facade
6. **Flyweight**
7. Composite
8. Bridge



Flyweight

- Use sharing to support large numbers of fine-grained objects efficiently.
- A flyweight is a shared object that can be used in multiple contexts simultaneously.
- It acts as an independent object in each context
  - It's indistinguishable from an instance of the object that's not shared.

Java API example: Wrapper classes like `java.lang.Integer`, `java.lang.Short`, `java.lang.Byte`, and `java.lang.Character`.



Flyweight

- The *intrinsic* state of a **FlyWeight** object is the part of the object state that is not dependent on the context in which it is used.
  - For example, a “char” attribute of a Character class representing a single letter.
- Conversely, the *extrinsic* state is the part that is context-dependent.
  - For example, the position of the character in the screen or in a text.



Flyweight

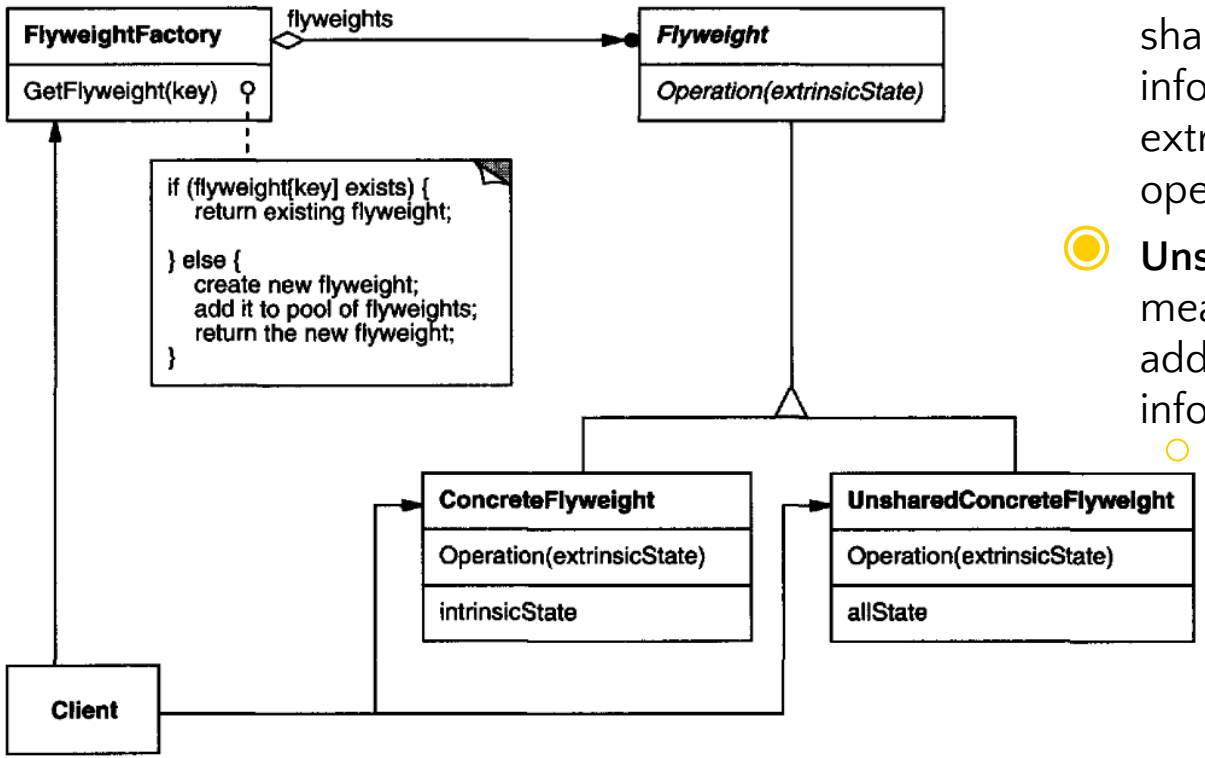
- Sharing of **FlyWeight** objects is made possible only if each **Flyweight** stores only intrinsic information.
- Because of this, all operations in **FlyWeights** must be provided with the relevant external extrinsic context.

Extrinsic

```
Character c= //… new flyweight for 'A'
Position p= …
c.printAt(p);
```

Intrinsic

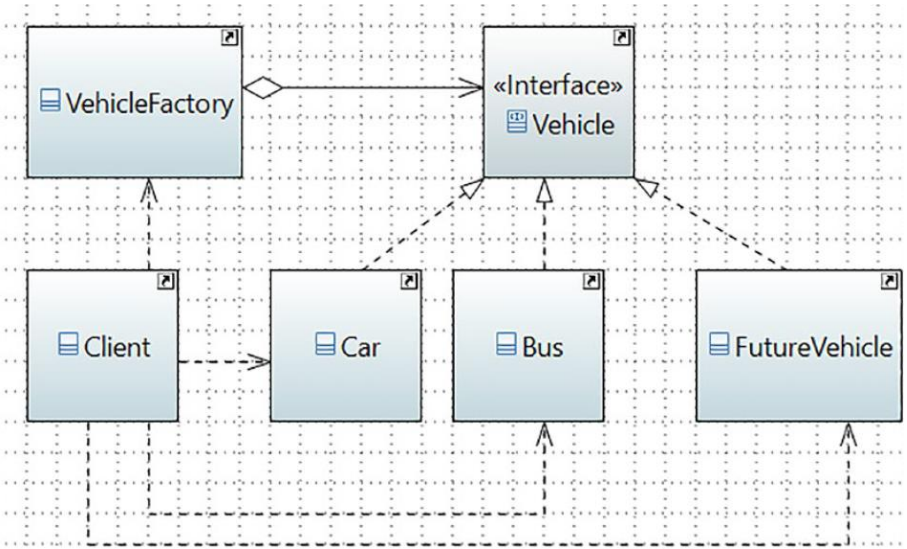
Flyweight



- The **FlyweightFactory** reuses previously created objects.
  - New objects are created only if a similar object was not created before.
- **ConcreteFlyweights**, meant to be shared, include only intrinsic information, and receive the extrinsic state as argument in operations
- **UnsharedConcreteFlyweights**, not meant to be shared, can include additional state beyond intrinsic information.
  - The extrinsic state received as argument will typically be ignored in this case



Flyweight



- Intrinsic vehicle information may include brand, model, and engine power.
- Extrinsic vehicle information may include owner, license plate number, and color.



Flyweight

- **Advantages:**
  - Saving storage space. The more flyweights are shared, and the larger the intrinsic start is, the larger are these savings.
  - If the extrinsic state is computed rather than stored, then additional savings are possible.
  - Because duplicated flyweight objects are not created, value-based identity comparisons can be replaced with object-based identity comparison.



Structural Patterns

1. Proxy
2. Decorator
3. Adapter
4. Facade
6. Flyweight
7. **Composite**
8. Bridge

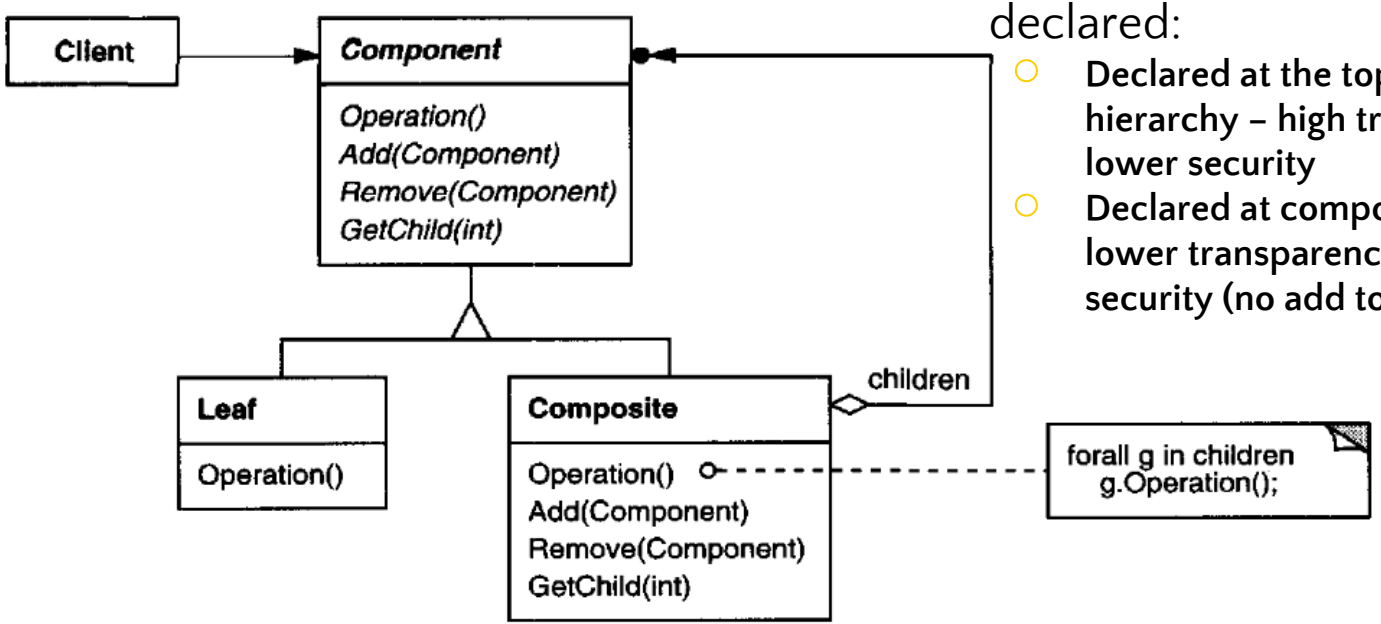


Composite

- It composes objects into tree structures to represent part-whole hierarchies.
- The Composite pattern lets clients treat individual objects and compositions of objects uniformly.

Java API example: Components of the AWT. For example, it is possible to add a new `java.awt.Component` to a `java.awt.Container` (itself also a `java.awt.Component`).

Composite

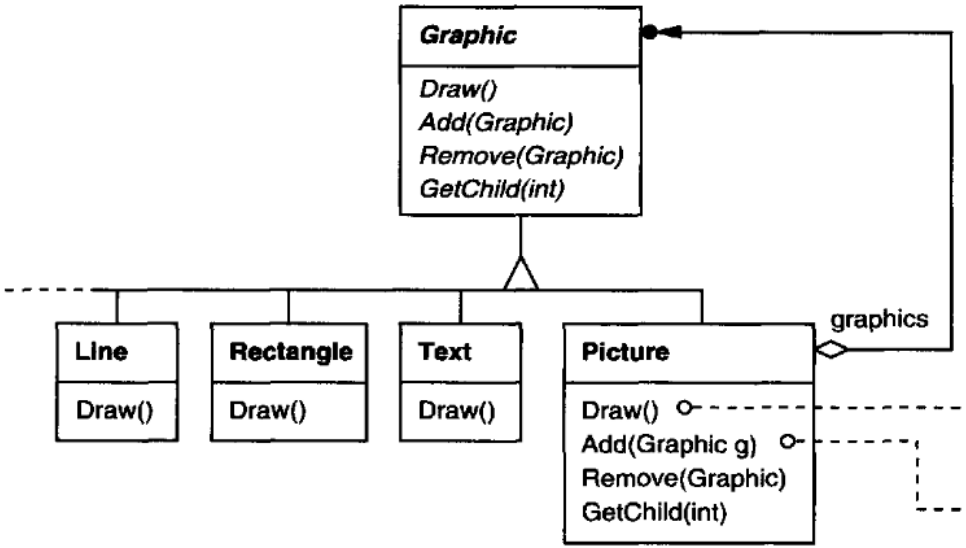


- There is a trade-off between transparency and safety, depending on the level at which the composite operations (add, etc) are declared:
  - **Declared at the top of hierarchy – high transparency, lower security**
  - **Declared at composite only – lower transparency, higher security (no add to leafs…)**

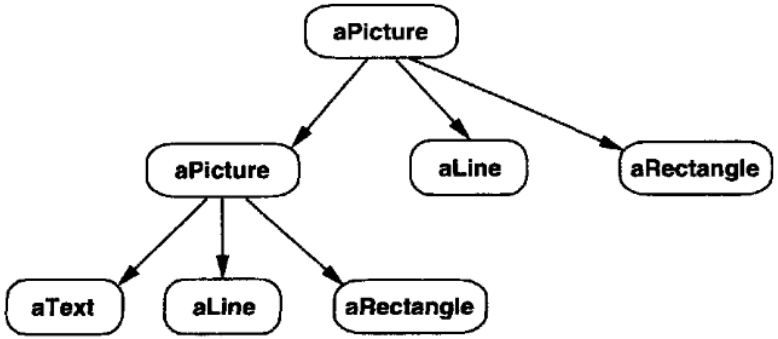


Composite

Class Definition



Object Hierarchy





Composite

- **Advantages:**
  - Suitable for representing part-whole hierarchies.
  - Possible to make clients ignore the difference between composite and non-composite objects and process them similarly.
  - Trade-off between transparency and security is possible.



Structural Patterns

1. Proxy
2. Decorator
3. Adapter
4. Facade
6. Flyweight
7. Composite
8. **Bridge**



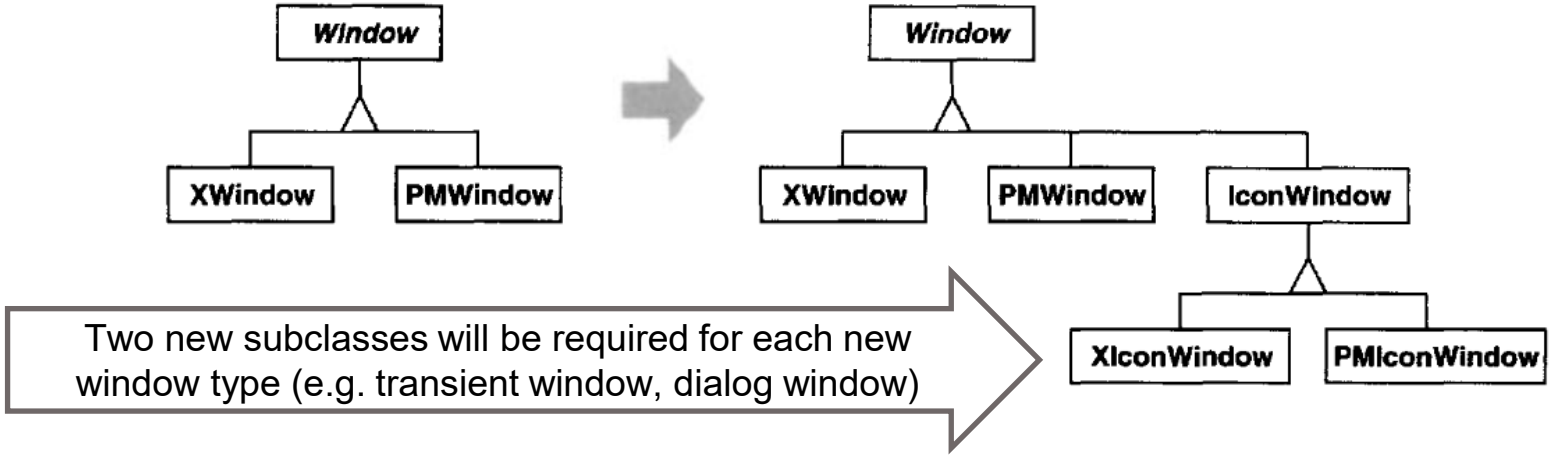
Bridge

- Decouple an abstraction from its implementation so that the two can vary independently.
  - Also known as Handle/Body.
- Decomposes a hierarchy into multiple hierarchies that can change independently.

Java API example: JDBC provides a bridge between your application and a particular database. For example, the `java.sql.DriverManager` class and the `java.sql.Driver` interface can form a bridge pattern where the first one plays the role of abstraction and the second one plays the role of implementors.

Bridge

- Inheritance binds the subclass statically to the abstraction, making it hard to modify, extend and reuse:

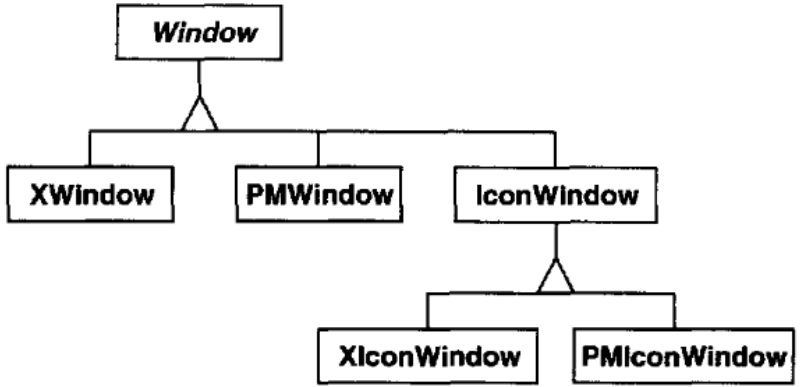




Bridge

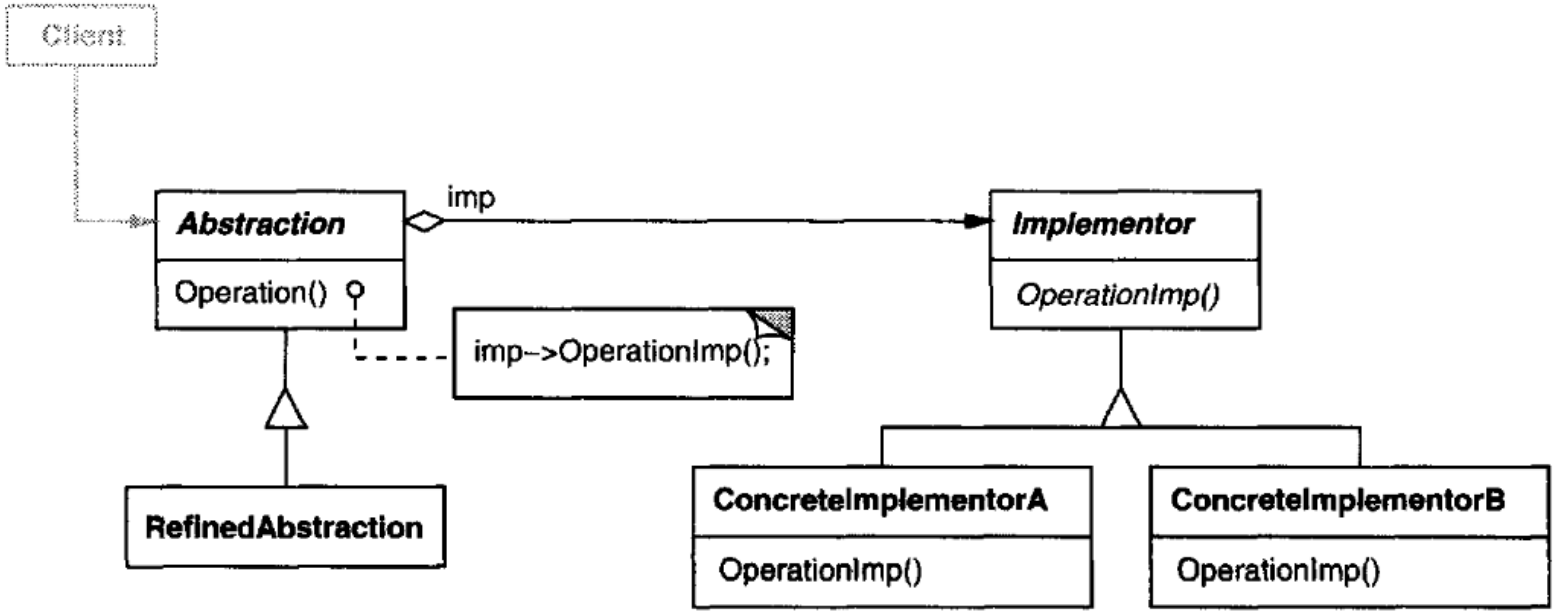
- Inheritance binds the subclass statically to the abstraction, making it hard to modify, extend and reuse:

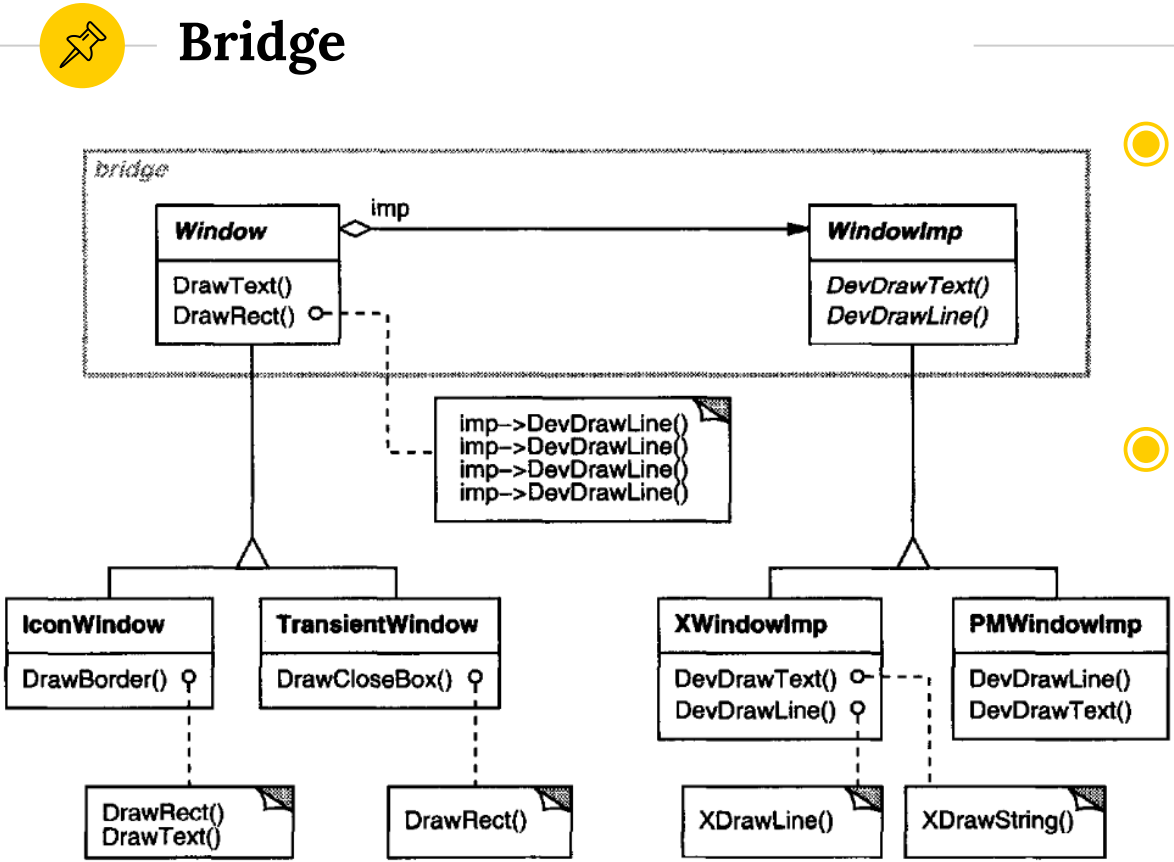
If a new type of system is considered (e.g. MSWindow), then a new corresponding subclass will have to be created for all other window types (IconWindow, DialogWindow, etc).





Bridge


Client
Abstraction
Operation()
imp
imp->OperationImp();
Implementor
OperationImp()
RefinedAbstraction
ConcreteImplementorA
OperationImp()
ConcreteImplementorB
OperationImp()



Bridge

- All operations on `Window` subclasses are implemented in terms of abstract operations from the `WindowImp` interface.
- This decouples the window abstractions from the various platform-specific implementations.



Bridge

- **Advantages:**
  - Implementations are not bound to abstractions.
  - The abstractions and implementations can grow independently.